

An EM-Based Multi-Task Ensemble Framework for Adaptive Concept Drift Detection in Data Streams

H. Sadoghi Yazdi, Department of Computer Engineering, Faculty of Engineering, Co

July 7, 2026

Abstract

The rapid growth of streaming data in dynamic environments necessitates classifiers that can efficiently handle concept drift while maintaining high predictive performance under strict resource constraints. Traditional single-model or fixed-ensemble approaches often fail to adapt quickly to various types of drift (abrupt, gradual, recurring, or incremental).

In this paper, we propose **EM-MTE**, an **Expectation-Maximization based Multi-Task Ensemble** framework for concept-drifting data streams. We model the streaming problem as a multi-task learning scenario where each drift-sensitive objective or data block/view is treated as a task. A shared library of online learning components (e.g., Perceptron, Passive-Aggressive, Confidence-Weighted models) is dynamically combined for each task through latent responsibility variables. These responsibilities are iteratively estimated via the EM algorithm, enabling principled, adaptive, and task-specific weighting of experts.

The proposed method provides a probabilistic interpretation of component combination, closed-form updates in the M-step, and natural handling of uncertainty in non-stationary environments. Extensive experiments on benchmark datasets from the UCI repository and real-world malicious web page detection demonstrate that EM-MTE achieves superior adaptation speed and accuracy compared to state-of-the-art streaming ensembles, while maintaining low memory and computational overhead.

Our framework bridges the gap between theoretical multi-task mixture modeling and practical online concept drift adaptation, offering robustness to label scarcity and different drift patterns.

Keywords: Concept drift; Data stream; Online learning; Multi-task learning; Ensemble method; Expectation-Maximization

1 Introduction

Optimal models for the data mining approach remain a challenging problem for computational intelligence researchers. Traditional methods for processing continuous data have several limitations: (i) data must be stored, (ii) processing is done on stored data based on appropriate parameters. In the static approach, the entire dataset is required which,

as time passes, causes computational challenges such as large memory usage, limited processing time, high dimensionality, and continuous change of infinite data [1]. Specific examples of data mining in the real world include image processing of satellite pictures, GPS systems, data security events such as spam detection systems, and information retrieval from text, etc.

Adapting to the behavior of arriving data is the most important feature of data stream approaches. These approaches track variations of the loaded data over time. In the process of data flow, a proper model must be trained on the current data to extract new unseen patterns. In online approaches, the feature weights of incoming data are updated to values suitable for prediction. Concept drift refers to a non-stationary and dynamic environment in which the distribution of data often changes over time [2]. In concept drift, there is no change in the marginal data distribution, but there is a difference in the conditional distribution of the target given the arriving input. A real-world example of concept drift occurs when the attention of users on a social network to some topic is changing. The conditional distribution of users' attention without change in the overall distribution is continuously shifting. There are many adaptive learning approaches aimed at reacting to concept drift; single and ensemble approaches are two main types [3].

In the single-base method, one model is used for the sequence of arriving data, whereas the ensemble method combines basic models simultaneously. Decision trees, sliding windows of sample-based learning, Naive Bayes, and neural networks are examples of the single-base method; AdaBoost, Random Forest, and cluster combiner methods are examples of ensemble-based methods. In single-base rather than ensemble-based approaches, complex models are often used to recognize the pattern of the data stream [4]. Several efficient classifiers can be applied to make ensemble models tractable for the streaming problem.

The importance of concept drift originates in the speed of adaptation needed to create a model. In this paper, because of the changing behavior of data over time and insufficient storage resources for the volume of data, online learning-based algorithms are used. Our system uses real-time modeling following the concept track. Our system can adapt to new behavior given the input data. Our principal research contributions are summarized as follows:

- Our system can adapt to the change in input data without the need for additional resources.
- Our system is capable of producing suitable real-time modeling when the volume of incoming data arrives quickly.
- To present an adaptive system, we propose an aggregation method based on the ensemble prediction of online experts.
- When we do not have an accurate label, our system tends to be robust against it.
- Our system was tested on benchmark and real datasets and compared to other existing models.

The rest of the paper is organized as follows: Section 2 reviews existing related work on concept drift. Section 3 demonstrates the proposed EM-based multi-task ensemble for data streams. Section 4 describes the implementation and experimental results. Finally, Section 5 presents the conclusion.

2 Related Work

In the real world, different types of concept drift occur. For example, in intrusion detection systems, attackers continuously change their strategies [13]. The core categories of algorithms for concept drift distribution can be stated as follows:

1. **Immediate drift:** recognizing a quick change of the current stream.
2. **Gradual drift:** unclear what kind of techniques are required for a re-duplicative bag of input data when there is a slow change in data distribution.
3. **Replicating drift:** when some patterns in the data stream repeat permanently.
4. **Development drift:** the incoming data stream pattern incrementally changes to another one.

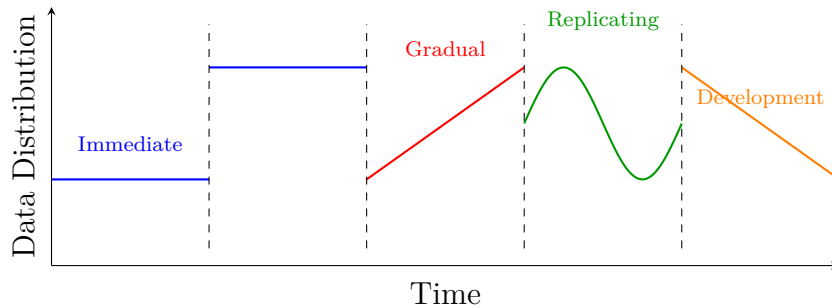


Figure 1: The types of concept drift in an incoming data stream.

Most of the works in stream processing contain sliding windows to display the concept drift [14, 15]. With sliding windows, we take an arriving data query group as the input to the model and then learn the optimal parameters for its concept drift [5]. Domingos et al. proposed a Very Fast Decision Tree (VFDT) for detecting sliding window changes in compound data for quick action recognition without the need for using large numbers of out-of-date data [6]. Hulten et al. proposed a system based on the CVFDT algorithm to cope with concept drift; they used a decision tree to model the streaming data, and whenever data change occurs, the newest subtree replaces the previous subtree and updates the node [7]. For determining the number of data by sliding windows, they used the Hoeffding bound; in other approaches, a fixed size is used to examine the newest nodes of the tree. The Hoeffding Adaptive Tree (HAT) and the Hoeffding Window Tree (HWT) are alternative versions of Hoeffding bounds for incrementally changing the number of sliding windows, proposed by Bifet [8]. Mena-Torres et al. proposed a similarity-based method called SimC to classify the input data [9]. In their system, an instance-based technique is applied in situations of implicit concept drift. SimC’s calculations are based on principles that: (i) each class of data must be grouped together by its attribute, (ii) the distance between each group is calculated, and (iii) older groups are eliminated by calculating age, which influences accuracy.

Other methods, such as cluster methods, have been used for adaptation. Tennant et al. proposed a new technique called Micro-Cluster Nearest Neighbor (MCNN) for classifying data streams. The structure of their system is based on the Nearest Neighbor (NN), which creates micro-clusters to continue adoption [10]. Each micro-cluster depends on the label of the cluster, an error threshold, and the performance of each micro-cluster.

Gama et al. introduced a method called DDM for combining online algorithms such as the perceptron, a neural network, and a decision tree; the feedback of each online learning algorithm is applied to the combination procedure [11]. Baena-García et al. proposed a new approach (EDDM) similar to DDM, with two differentiating factors: (i) they employ both online learning and batch algorithms, and (ii) similar to DDM, the combination is based on the distance of each classification error [12]. Ross et al. chose the classification error rate for detecting concept drift, using the Exponentially Weighted Moving Average (EWMA) [5]. In their system, an online classifier is applied to the investigation of data features for a generalized solution to the concept drift problem. EWMA represents concept drift based on a sequence of Bernoulli random variables.

3 The Proposed Method

A significant challenge in streaming data is to design an adaptive ensemble that can handle various types of concept drift with limited memory and computational resources. To address this, we propose **EM-MTE** (Expectation-Maximization based Multi-Task Ensemble), which reformulates concept drift adaptation as a multi-task mixture learning problem.

The proposed framework maintains a shared library of online learners (components) that are dynamically combined for different tasks through latent responsibility variables estimated by the Expectation-Maximization (EM) algorithm. This provides a principled probabilistic approach for adaptation, going beyond heuristic weighting methods such as Dynamic Weighted Majority (DWM).

Figure 2 presents the overall architecture of the EM-MTE framework. The remainder of this section details the problem formulation, the mixture model, the EM procedure, and the online adaptation mechanism.

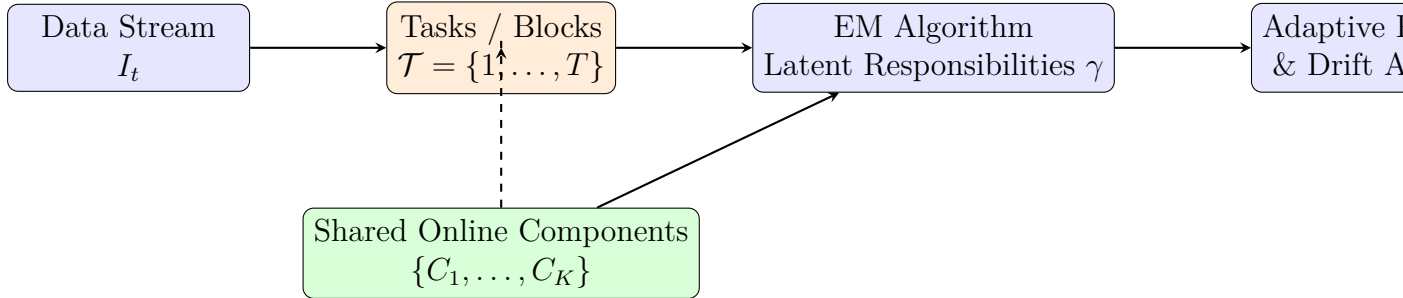


Figure 2: Overall architecture of the proposed EM-MTE framework.

3.1 Problem Setup and Notation

We consider a data stream classification problem where instances arrive sequentially over time. Let the input data stream be denoted as:

$$D = \{(I_t, L_t) \mid t = 1, 2, \dots\},$$

where $I_t \in \mathbb{R}^d$ is the feature vector of the t -th instance and $L_t \in \{-1, +1\}$ (or $\{1, \dots, C\}$ for multi-class) is the corresponding true label.

Due to the potentially infinite nature of the stream and memory constraints, we cannot store all historical data. Instead, we process the stream in sequential blocks (or windows) $S = \{s_1, s_2, \dots, s_B\}$. Each block s_b can be viewed as a temporary task or context that may undergo concept drift.

Following the multi-task learning perspective, we define a set of tasks $\mathcal{T} = \{1, 2, \dots, T\}$, where each task $r \in \mathcal{T}$ corresponds to a specific data block, temporal view, or drift-sensitive objective. For each task r , the associated dataset is:

$$\mathcal{D}^{(r)} = \{(x_i^{(r)}, y_i^{(r)})\}_{i=1}^{N_r},$$

where $x_i^{(r)}$ is derived from the incoming instances I_t , and $y_i^{(r)}$ is the label or a task-specific target.

We maintain a shared library of K online learning components (experts) $\mathcal{C} = \{C_1, C_2, \dots, C_K\}$. Each component C_k is an incremental learner that can be updated online. Examples of such components include Perceptron, Passive-Aggressive (PA), Online Gradient Descent (OGD), and Confidence-Weighted (CW) models.

Let θ_k denote the parameters of component C_k . The goal is to dynamically combine these shared components for each task while adapting to concept changes with minimal delay and resource usage.

3.2 Task-Conditioned Mixture Model

The core idea of our approach is to model the prediction for each task as a *mixture* of the shared online components. Formally, the conditional probability of the label given the input for task r is expressed as:

$$p(y_i^{(r)} | x_i^{(r)}) = \sum_{k=1}^K \pi_k^{(r)} \cdot p_k(y_i^{(r)} | x_i^{(r)}, \theta_k), \quad (1)$$

where:

- $\pi_k^{(r)} \geq 0$ is the mixing coefficient (weight) of component k for task r , satisfying $\sum_{k=1}^K \pi_k^{(r)} = 1$.
- $p_k(y_i^{(r)} | x_i^{(r)}, \theta_k)$ is the predictive distribution of the k -th online component with parameters θ_k .

This formulation treats each task as having its own *soft* combination of experts, allowing flexible adaptation to local concept drift while benefiting from knowledge sharing across tasks through the common component library $\mathcal{C}$.

To make the assignment of samples to components explicit, we introduce latent responsibility variables $z_{ik}^{(r)} \in \{0, 1\}$ indicating whether component k is responsible for the i -th sample of task r . These latent variables enable a probabilistic interpretation of how the model detects and adapts to different types of concept drift (abrupt, gradual, recurring, or incremental).

This mixture modeling approach goes beyond heuristic weighting schemes such as Dynamic Weighted Majority (DWM) by providing a principled maximum likelihood estimation framework via the Expectation-Maximization (EM) algorithm.

3.3 Latent Responsibility Variables and EM Framework

To enable principled adaptation, we introduce latent responsibility variables that explicitly model the contribution of each component to individual samples and tasks. For each sample i of task r , let $z_{ik}^{(r)} \in \{0, 1\}$ be a latent indicator variable such that $z_{ik}^{(r)} = 1$ if component C_k is responsible for generating $(x_i^{(r)}, y_i^{(r)})$, and $\sum_{k=1}^K z_{ik}^{(r)} = 1$.

The complete-data log-likelihood for the mixture model can then be written as:

$$\mathcal{L}_c = \sum_{r=1}^T \sum_{i=1}^{N_r} \sum_{k=1}^K z_{ik}^{(r)} \left[\log \pi_k^{(r)} + \log p_k(y_i^{(r)} | x_i^{(r)}, \theta_k) \right]. \quad (2)$$

Since the latent variables $z_{ik}^{(r)}$ are unobserved, we maximize the *expected* complete-data log-likelihood using the Expectation-Maximization (EM) algorithm. This approach offers several theoretical and practical advantages over heuristic methods (e.g., DWM):

- **Probabilistic Interpretation:** Responsibilities $\gamma_{ik}^{(r)} = \mathbb{E}[z_{ik}^{(r)}]$ provide a soft assignment that naturally captures uncertainty and gradual concept drift.
- **Optimality:** EM iteratively finds a local maximum of the marginal likelihood, offering a more principled optimization compared to ad-hoc weight updates.
- **Shared Knowledge with Task Adaptation:** Components are shared globally (θ_k), while mixing weights $\pi_k^{(r)}$ are task-specific, reducing negative transfer while allowing rapid adaptation to local drift.
- **Handling Non-Stationarity:** The framework can detect changes in responsibility distributions over time, serving as an implicit drift detector.

The EM procedure consists of two main steps that will be detailed in the following subsections. This formulation draws inspiration from mixture of experts models and the EM-SVDD framework, extending them to the challenging online streaming setting.

3.4 E-step and M-step Updates

The EM algorithm alternates between the following two steps:

E-step (Expectation): Given the current parameter estimates $\Theta^{(t)} = \{\theta_k^{(t)}, \pi_k^{(r,t)}\}$, compute the posterior responsibilities:

$$\gamma_{ik}^{(r)} = \frac{\pi_k^{(r)} p_k(y_i^{(r)} | x_i^{(r)}, \theta_k)}{\sum_{j=1}^K \pi_j^{(r)} p_j(y_i^{(r)} | x_i^{(r)}, \theta_j)}. \quad (3)$$

M-step (Maximization): Update the parameters by maximizing the expected complete-data log-likelihood:

$$\pi_k^{(r)} \propto \sum_{i=1}^{N_r} \gamma_{ik}^{(r)}, \quad (4)$$

$$\theta_k \leftarrow \arg \max_{\theta_k} \sum_{r=1}^T \sum_{i=1}^{N_r} \gamma_{ik}^{(r)} \log p_k(y_i^{(r)} | x_i^{(r)}, \theta_k). \quad (5)$$

For many online components (e.g., Perceptron, PA, CW), the M-step admits efficient incremental updates, making the algorithm suitable for streaming scenarios.

3.5 Online Adaptation and Drift Handling

In the streaming setting, we process data blocks sequentially. When a new block arrives, responsibilities are updated incrementally. Now, we decouple the sources of error into optimization error, gradient noise, and drift-induced error.

The EM-MTE framework naturally handles different drift regimes:

- **High drift-to-noise regime:** Larger mixing weight updates allow fast tracking of abrupt changes.
- **Low drift-to-noise regime:** More stable responsibilities with gradual adaptation, similar to step decay schedules in drifting optimization.
- **Recurring drift:** Previously useful components can regain high responsibility without restarting the entire ensemble.

Significant shifts in the responsibility distribution $\pi^{(r)}$ across blocks serve as a signal of concept drift, providing an implicit and probabilistic drift detection mechanism.

3.6 Algorithm

Algorithm 1 EM-MTE: EM-based Multi-Task Ensemble for Streaming Data

- 1: **Input:** Data stream, number of components K , tasks $\mathcal{T}$
 - 2: Initialize shared components $\{C_k\}$ and mixing weights $\pi_k^{(r)}$
 - 3: **for** each new data block s_b **do**
 - 4: Assign block to task r (or create new task)
 - 5: **for** each sample (x_i, y_i) in block **do**
 - 6: **E-step:** Compute responsibilities $\gamma_{ik}^{(r)}$ using Eq. (3)
 - 7: **M-step:** Update $\pi_k^{(r)}$ and incrementally update θ_k
 - 8: **end for**
 - 9: Adapt predictions using weighted combination of components
 - 10: **end for**
-

3.7 Complexity Analysis

Let K be the number of components and d the feature dimension. The per-sample computational cost is $\mathcal{O}(K \cdot c)$, where c is the update cost of a single component (typically $\mathcal{O}(d)$). Memory complexity is $\mathcal{O}(K \cdot m + T \cdot K)$, where m is the memory per component.

In terms of tracking error under drift, our method achieves a favorable trade-off between adaptation speed and stability by dynamically adjusting responsibilities rather than using fixed step sizes or heuristic weight updates. This leads to better performance in both high and low drift-to-noise regimes compared to purely heuristic ensembles.

A significant issue in concept drift detection is the reliability of functional algorithms with minimum delay and storage usage. In this paper, the proposed method is reformulated as an *EM-based multi-task ensemble*. The central idea is that a data stream

is not handled by one global online model or by a fixed averaging of experts. Instead, each drift-sensitive prediction objective is treated as a task, and each task learns a soft combination of shared online components. The combination weights are estimated by the Expectation-Maximization (EM) principle through latent responsibility variables.

We assume that the incremental input stream constructs the dataset D where:

$$D = \{(I_t; L_t) \mid I_t \in \mathbb{R}^d, L_t \in \{-1, +1\}, t = 1, 2, \dots, n\}$$

and I_t is each incoming instance at time t and L_t is its label. The stream is divided into sequential blocks $S = \{s_1, \dots, s_B\}$. Each block, drift type, label view, or application objective can define a task. We denote the set of tasks by $\mathcal{T} = \{1, \dots, T\}$ and the dataset assigned to task r by

$$\mathcal{D}^{(r)} = \{(x_i^{(r)}, y_i^{(r)})\}_{i=1}^{N_r}.$$

In the streaming case, $x_i^{(r)}$ is obtained from I_t , and $y_i^{(r)}$ is the corresponding label or task-specific target. The method uses K shared components $\{C_1, \dots, C_K\}$. A component can be a linear-order learner, such as Perceptron, OGD, and Passive-Aggressive models, or a Gaussian-order learner, such as CW, AROW, SOP, and related confidence-weighted models. These components form an ensemble library, but the ensemble is not fixed: every task has its own mixture weights over the same library.

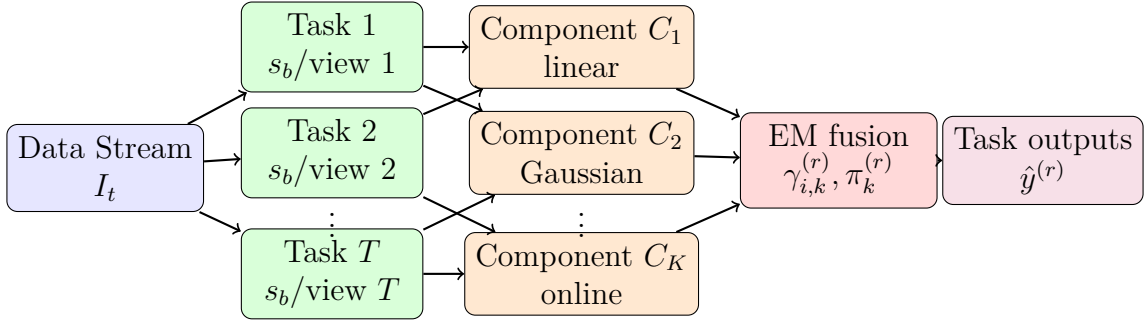


Figure 3: Overall view of the proposed EM-based multi-task ensemble. Tasks are routed to a shared component library, and EM estimates task-specific responsibilities and mixture weights.

3.8 Online Components as Shared Task Solvers

Perceptron is an online algorithm based on the feedback of one hidden layer for streaming data [16]. Another similar approach is OGD (Online Gradient Descent), which is based on minimizing the gradient of incoming data [17]. The weaknesses of Perceptron and OGD are in identifying incremental and sudden drift, respectively. The use of different types of Passive-Aggressive (PA) algorithms can be considered to improve concept drift detection [18]. In the proposed model, these online learners are not independent final classifiers; they are shared components that can be reused by several tasks with different responsibilities. For a PA component, the best weight parameter in every iteration is obtained by solving

$$w_{t+1} = \arg \min_{W \in \mathbb{R}^d} \frac{1}{2} \|W - w_t\|^2 \quad \text{s.t.} \quad L_t(w_t \cdot I_t) \geq 1 \quad (6)$$

The loss function and input value are important driving factors for determining the model. In each round, the model trends towards the best weight incrementally. By solving the optimization problem in Eq. (6), PA updates its model as follows:

$$w_{t+1} = w_t + \frac{l_t}{\|I_t\|^2} \gamma L_t I_t \quad (7)$$

where w_{t+1} is the new weight based on input data stream; $l_t = \max(0, 1 - L_t w_t \cdot I_t)$ is the hinge loss; and γ is a parameter which determines the importance of the loss function. Various states of concept drift can be considered using the cost function and other parameters. Types of *linear-order* online learners for concept drift are shown in Table 1.

Table 1: Types of *Linear-order* online learning algorithms and their most impactful aspect for concept drift

Algorithm	Loss function	Weight update	Concept drift
Perceptron [16]	$l_t = \max(0, 1 - L_t w_t I_t)$	$w_{t+1} = w_t + l_t L_t I_t$	Immediate
OGD (logistic) [17]	$l_t = \log(1 + e^{-L_t w_t I_t})$	$w_{t+1} = w_t + \gamma l_t L_t I_t$	Replicating
PA [18]	$l_t = \max(0, 1 - L_t w_t I_t)$	$w_{t+1} = w_t + \frac{l_t}{\ I_t\ ^2} \gamma L_t I_t$	Gradual
PA-I [18]	$l_t = \max(0, 1 - L_t w_t I_t)$	$\gamma_t = \min\left(C, \frac{l_t}{\ I_t\ ^2}\right); \quad w_{t+1} = w_t + \gamma_t L_t I_t$	Gradual, Replicating
PA-II [18]	$l_t = \max(0, 1 - L_t w_t I_t)$	$\gamma_t = \frac{l_t}{\ I_t\ ^2 + \frac{1}{p^2}}; \quad w_{t+1} = w_t + \gamma_t L_t I_t$	Gradual, Replicating
ROMMA [19]	$l_t = \max(0, 1 - L_t w_t I_t)$	$\alpha_t = \frac{(\ I_t\ \cdot \ w\ ^2 - L_t(w \cdot I_t))}{(\ I_t\ \cdot \ w\)^2 - (w \cdot I_t)^2}; \quad \beta_t = \frac{\ w\ ^2 \cdot L_t(w \cdot I_t)}{(\ I_t\ \cdot \ w\)^2 - (w \cdot I_t)^2}; \quad w_{t+1} = \alpha_t w_t + \beta_t I_t$	Immediate, Replicating
aROMMA [19]	$l_t = 1 - L_t w_t I_t$	(same as ROMMA)	Immediate, Replicating
ALMA [20]	$l_t = (1 - \alpha) \frac{1}{\alpha} \frac{1}{p \sqrt{p}} \frac{1}{\sqrt{k+1}} - L_t w_t I_t$	$\beta_t = \frac{C}{p - \frac{1}{p} \sqrt{p}}; \quad w_{t+1} = w_t + \beta_t L_t I_t; \quad w_{t+1} \leftarrow \frac{w_{t+1}}{\max(1, \ w_{t+1}\)}$	Replicating

The *Gaussian-order* is the other type of online learning algorithm, which updates parameters based on a Gaussian distribution. The mean vector $\mu \in \mathbb{R}^d$ and covariance matrix $\Sigma \in \mathbb{R}^{d \times d}$ of the input data distribution are the principal factors in determining concept drift. Confidence Weighting (CW) is one of the most popular algorithms for the Gaussian-order [21]. For data streams where we have low variance of the obtained data, CW determines more confidence in the average data. To recognize concept drift in a data stream, the Kullback-Leibler (KL) divergence is used as follows:

$$(\mu_{t+1}, \Sigma_{t+1}) = \arg \min_{(\mu, \Sigma)} D_{\text{KL}}(\mathcal{N}(\mu, \Sigma) \parallel \mathcal{N}(\mu_t, \Sigma_t)) \quad \text{s.t.} \quad L_t(\mu^\top I_t) \geq 1 \quad (8)$$

After solving the cost function in Eq. (8), the new weighting of each input data is calculated as follows:

$$w_{t+1} = w_t + \alpha_t L_t I_t \cdot \Sigma_t \quad (9)$$

$$\Sigma_{t+1} = \Sigma_t - \beta_t I_t \Sigma_t^\top \quad (10)$$

In these algorithms, the significant parameters α_t and β_t depend on the weight and Gaussian distribution of input features, respectively. These are more efficient for computing the loss function. Other types of Gaussian-order online experts are shown in Table 2.

Table 2: Types of *Gaussian-order* online learning algorithms

Algorithm	Loss function	Weight update	Concept drift
CW [21]	$l_t = \max(0, 1 - L_t w_t I_t)$	$\alpha_t = \frac{-\phi_t + \sqrt{\phi_t^2 + 4w_t}}{2w_t}; w_{t+1} = w_t + \alpha_t L_t I_t \Sigma_t; \Sigma_{t+1} = \Sigma_t - \beta_t (\Sigma_t I_t)^2$	Gradual, Replicating, Development
SCW-I [22]	$l_t = \phi \sqrt{v_t} - L_t w_t I_t$	$\alpha_t = \max\left(0, \frac{-(\phi^2 I_t v_t + \phi w_t \sqrt{\phi^2 I_t^2 + 4w_t})}{2(\phi^2 I_t^2 - I_t^2)}\right); \alpha_{t+1} = \min(\alpha_t, C); w_{t+1} = w_t + \alpha_{t+1} L_t I_t \Sigma_t; \Sigma_{t+1} = \Sigma_t - \beta_t (\Sigma_t I_t)^2$	Gradual, Replicating, Development
SCW-II [22]	$l_t = \phi \sqrt{v_t} - L_t w_t I_t$	$\lambda = \sqrt{(\phi^2 v_t^2 + I_t^2) \cdot C}; \alpha_t = \max(0, \frac{-\phi^2 v_t I_t + \lambda}{\phi^2 v_t + \frac{1}{2C}}); \alpha_{t+1} = \min(\alpha_t, C); w_{t+1}, \Sigma_{t+1}$ same as SCW-I	Gradual, Replicating, Development
AROW [23]	$l_t = \max(0, 1 - L_t w_t I_t)$	$\alpha_t = \frac{I_t}{I_t^T \Sigma_t I_t + \gamma}; w_{t+1} = w_t + \alpha_t L_t \Sigma_t I_t; \Sigma_{t+1} = \Sigma_t - \frac{\Sigma_t I_t I_t^T \Sigma_t}{I_t^T \Sigma_t I_t + \gamma}$	Gradual, Replicating, Development
NAROW [23]	$l_t = 1 - L_t w_t I_t$	$\beta_t = \frac{1}{I_t^T \Sigma_t I_t + \frac{1}{2}}; \alpha_t = \max(0, I_t) \beta_t; w_{t+1} = w_t + \alpha_t L_t \Sigma_t I_t; \Sigma_{t+1} = \Sigma_t - \beta_t \Sigma_t I_t I_t^T \Sigma_t$	Gradual, Replicating, Development
SOP [24]	$l_t = \max(0, 1 - L_t w_t I_t)$	$\beta_t = \frac{1}{I_t^T \Sigma_t I_t + 1}; \alpha_t = \max(0, I_t) \beta_t; w_{t+1} = w_t + \alpha_t L_t I_t; \Sigma_{t+1} = \Sigma_t - \beta_t (\Sigma_t I_t)^2$	Gradual, Replicating, Development
NHERD [25]	$l_t = 1 - L_t w_t I_t$	$\beta_t = \frac{1}{I_t^T \Sigma_t I_t + \gamma}; \alpha_t = \max(0, I_t) \beta_t; w_{t+1} = w_t + \alpha_t L_t \Sigma_t I_t; \Sigma_{t+1} = \Sigma_t - \beta_t^2 (I_t^T \Sigma_t I_t + 2\gamma) (\Sigma_t I_t)^2$	Gradual, Replicating, Development
IELLIP [26]	$l_t = \max(0, 1 - L_t w_t I_t)$	$\alpha_t = \frac{I_t}{\sqrt{I_t^T \Sigma_t I_t}}; \beta_t = \frac{L_t \Sigma_t I_t}{\sqrt{I_t^T \Sigma_t I_t}}; w_{t+1} = w_t + \alpha_t \beta_t; \Sigma_{t+1} = \frac{\Sigma_t - \gamma \beta_t \beta_t^T}{1 - \gamma}$	Gradual, Replicating, Immediate

3.9 EM-Based Multi-Task Ensemble Formulation

The proposed system, denoted by EM-MTE, models the output distribution of each task as a mixture over the shared online components:

$$p(y_i^{(r)} | x_i^{(r)}) = \sum_{k=1}^K \pi_k^{(r)} p(y_i^{(r)} | x_i^{(r)}, \theta_k), \quad (11)$$

where θ_k denotes the parameters of component C_k , and $\pi_k^{(r)}$ is the task-specific mixture weight of component k for task r . The constraints are $\pi_k^{(r)} \geq 0$ and $\sum_{k=1}^K \pi_k^{(r)} = 1$. This equation is the ensemble part of the method: the final prediction is a task-specific weighted combination of component predictions. It is also the multi-task part: all tasks use the same component library $\{\theta_k\}_{k=1}^K$, but each task is allowed to use a different mixing vector $\boldsymbol{\pi}^{(r)}$.

For each task-sample pair (r, i) , we introduce a latent variable

$$z_i^{(r)} \in \{1, \dots, K\},$$

where $z_i^{(r)} = k$ means that component C_k is responsible for explaining sample i of task r . The complete-data likelihood is

$$\mathcal{L}_c(\Theta) = \prod_{r=1}^T \prod_{i=1}^{N_r} \prod_{k=1}^K \left[\pi_k^{(r)} p(y_i^{(r)} | x_i^{(r)}, \theta_k) \right]^{\mathbb{I}(z_i^{(r)}=k)}, \quad (12)$$

and the complete-data log-likelihood is

$$\log \mathcal{L}_c(\Theta) = \sum_{r=1}^T \sum_{i=1}^{N_r} \sum_{k=1}^K \mathbb{I}(z_i^{(r)} = k) \left[\log \pi_k^{(r)} + \log p(y_i^{(r)} | x_i^{(r)}, \theta_k) \right]. \quad (13)$$

For a margin-based binary learner, the likelihood can be written as

$$p(y_i^{(r)} | x_i^{(r)}, \theta_k) = \sigma(y_i^{(r)} f_k(x_i^{(r)}; \theta_k)), \quad \sigma(a) = \frac{1}{1 + \exp(-a)}, \quad (14)$$

or equivalently by an exponential loss-based likelihood

$$p(y_i^{(r)} | x_i^{(r)}, \theta_k) \propto \exp \left[-\ell_k(\theta_k; (x_i^{(r)}, y_i^{(r)})) \right]. \quad (15)$$

Thus, a component with lower task-specific loss receives a larger posterior responsibility.

3.10 E-Step and M-Step

Given the current parameters at EM iteration q , the posterior responsibility of component k for sample i of task r is

$$\gamma_{i,k}^{(r,q)} = \frac{\pi_k^{(r,q)} p(y_i^{(r)} | x_i^{(r)}, \theta_k^{(q)})}{\sum_{j=1}^K \pi_j^{(r,q)} p(y_i^{(r)} | x_i^{(r)}, \theta_j^{(q)})}. \quad (16)$$

In loss form, Eq. (16) becomes

$$\gamma_{i,k}^{(r,q)} = \frac{\pi_k^{(r,q)} \exp[-\ell_k(\theta_k^{(q)}; (x_i^{(r)}, y_i^{(r)}))]}{\sum_{j=1}^K \pi_j^{(r,q)} \exp[-\ell_j(\theta_j^{(q)}; (x_i^{(r)}, y_i^{(r)}))]} \quad (17)$$

This is the key difference from a conventional ensemble. A fixed ensemble assigns one global weight to each expert, while EM-MTE assigns a task- and sample-dependent responsibility. Therefore, the same component may be highly responsible for an immediate-drift task and weakly responsible for a gradual-drift task.

The expected complete-data log-likelihood is

$$Q(\Theta | \Theta^{(q)}) = \sum_{r=1}^T \sum_{i=1}^{N_r} \sum_{k=1}^K \gamma_{i,k}^{(r,q)} \left[\log \pi_k^{(r)} + \log p(y_i^{(r)} | x_i^{(r)}, \theta_k) \right]. \quad (18)$$

For fixed component parameters, maximizing Q with respect to $\pi_k^{(r)}$ under the simplex constraint yields

$$\pi_k^{(r,q+1)} = \frac{1}{N_r} \sum_{i=1}^{N_r} \gamma_{i,k}^{(r,q)}. \quad (19)$$

The component parameters are updated by solving

$$\theta_k^{(q+1)} = \arg \min_{\theta_k} \sum_{r=1}^T \sum_{i=1}^{N_r} \gamma_{i,k}^{(r,q)} \ell_k(\theta_k; (x_i^{(r)}, y_i^{(r)})) + \Omega_k(\theta_k), \quad (20)$$

where Ω_k is the regularization term of component C_k . For online components, Eq. (20) is implemented as a responsibility-weighted online update. For example, a PA-type update becomes

$$w_{t+1}^{(k)} = w_t^{(k)} + \gamma_{t,k}^{(r)} \frac{l_t^{(k)}}{\|I_t\|^2} L_t I_t. \quad (21)$$

Hence, tasks do not merely vote over experts; they actively determine how strongly each component is trained and reused.

3.11 Ensemble Prediction and Algorithm

After EM updates the task-specific mixture vector, the score of task r for a new instance x is computed by

$$F^{(r)}(x) = \sum_{k=1}^K \pi_k^{(r)} f_k(x; \theta_k), \quad (22)$$

and the binary prediction is

$$\hat{y}^{(r)} = \text{sign}(F^{(r)}(x)). \quad (23)$$

For probabilistic classification, the prediction probability is

$$P(y = 1 \mid x, r) = \sum_{k=1}^K \pi_k^{(r)} \sigma(f_k(x; \theta_k)). \quad (24)$$

This formulation preserves the ensemble advantage of combining diverse learners, while allowing different tasks to combine them differently.

Algorithm 2 EM-Based Multi-Task Ensemble (EM-MTE)

Require: Tasks $\{\mathcal{D}^{(r)}\}_{r=1}^T$; components $\{C_k\}_{k=1}^K$; stream blocks $\{s_b\}_{b=1}^B$

Ensure: Task predictions $\hat{y}^{(r)}$ and updated parameters $\{\theta_k, \pi_k^{(r)}\}$

- 1: Initialize component parameters $\theta_k^{(0)}$ for $k = 1, \dots, K$
 - 2: Initialize $\pi_k^{(r,0)} \leftarrow 1/K$ for all tasks and components
 - 3: **for** each block s_b **do**
 - 4: Assign samples in s_b to one or more tasks $\mathcal{D}^{(r)}$
 - 5: **for** each EM iteration $q = 0, 1, \dots, Q - 1$ **do**
 - 6: **for** each task r and sample i **do**
 - 7: Compute $\gamma_{i,k}^{(r,q)}$ for all k using Eq. (16) or Eq. (17)
 - 8: **end for**
 - 9: **for** each task r and component k **do**
 - 10: Update $\pi_k^{(r,q+1)}$ using Eq. (19)
 - 11: **end for**
 - 12: **for** each component k **do**
 - 13: Update $\theta_k^{(q+1)}$ with Eq. (20)
 - 14: **end for**
 - 15: **end for**
 - 16: Predict each task by Eqs. (22)–(23)
 - 17: **end for**
-

With respect to streaming concept drift, sliding windows are used to determine when the task mixture weights should be refreshed. As shown in Figure 4, the responsibilities and weights learned from the preceding block s_{k-1} guide the predictions in the current block s_k . When data stream S arrives, it is split into small sequential blocks s_k such that $S = s_1, s_2, \dots, s_m$. The EM update then replaces a hand-crafted fixed ensemble rule with a likelihood-based task-combination rule.

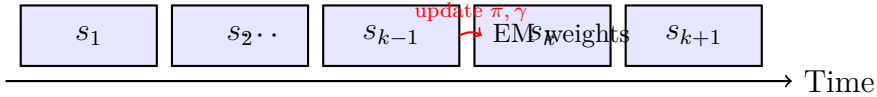


Figure 4: The block-wise EM update for obtaining task-specific mixture weights in a data stream.

Definition 1. Dataset D is an infinite sequence of data that is continually changing. For every sample $\vec{d}_i \in D$, it follows the schema $\vec{d}_i = \langle f_1, f_2, \dots, f_d \rangle$ with d -dimensional features. The proposed EM-MTE model can apply a new feature without interrupting prediction. When a new feature f_{d+1} is extracted, its initial value equals zero: $\langle f_1, f_2, \dots, f_d, 0 \rangle$.

Definition 2. (*Task-Component Responsibility Matrix*). For each task r and stream block s_b , the matrix

$$\Gamma^{(r,b)} = \left[\gamma_{i,k}^{(r)} \right]_{N_r \times K}$$

stores the posterior contribution of component C_k to sample i of task r . This matrix is the EM-based replacement for a purely correlation-based expert matrix: it uses both the current task mixture $\pi_k^{(r)}$ and the component likelihood or loss.

The block prediction can still be written as a weighted ensemble:

$$P^{(r)}(\boldsymbol{\pi}) = \sum_{k=1}^K \pi_k^{(r)} C_k(x), \quad (25)$$

where $C_k(x)$ is the prediction value of component k . However, the weights are now estimated by EM rather than by minimizing only a covariance-based error-correlation objective. If desired, the previous correlation structure can be retained as a diversity regularizer:

$$\mathcal{J} = -Q(\Theta \mid \Theta^{(q)}) + \rho \sum_{r=1}^T (\boldsymbol{\pi}^{(r)})^\top \Sigma_e^{(r)} \boldsymbol{\pi}^{(r)}, \quad (26)$$

where $\Sigma_e^{(r)}$ is the error covariance matrix of the components for task r and $\rho \geq 0$ controls the influence of diversity. When $\rho = 0$, the method is the pure EM-based multi-task ensemble. When $\rho > 0$, highly correlated error patterns are discouraged, which preserves the robustness motivation of the original ensemble model.

To classify between legitimate and malicious web pages, we use the sigmoid function as shown in Figure 5. The classification is based on the EM-weighted output values of the proposed model: for legitimate web pages the probability is greater than the decision threshold and for malicious web pages it is lower.

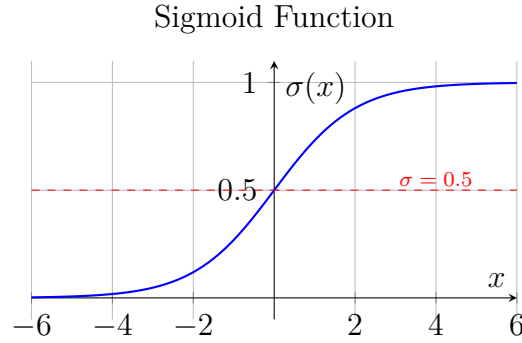


Figure 5: Sigmoid function used to classify between legitimate and malicious web pages.

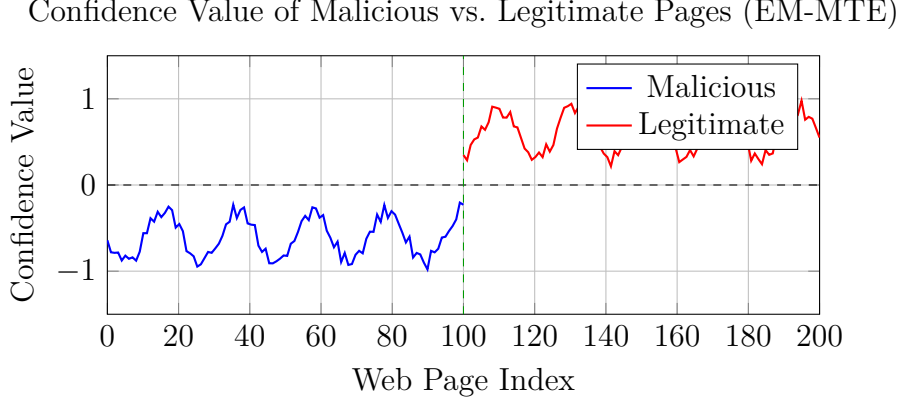


Figure 6: The confidence value of malicious and legitimate web pages classified by the proposed EM-MTE model.

4 Evaluation Configuration

We evaluated the performance of our model on benchmark and real datasets, then compared it to other simple and ensemble systems for detection. To circumvent the streaming concept drift problems, we present two types of online learners and evaluate each of them. Some Linear-order and Gaussian-order learners can use a recursive form, but in our work a streaming type is employed where data inputs arrive one at a time. Basically, the linear-order learner is chosen to cope with immediate and development drift, but most of the time the Gaussian-order learner is applied to detect development, gradual, and replicating concept drift. Both types of online learners are suitable for computational processing and time consumption. The proposed system is based on combining the online learner views about each data observation block, which influences the next block.

Some parameters in our system need to be initialized by the user. To relax such constraints on fundamental parameters, we apply random initial values, set only once, so that the online learning objective function can converge to the best result where all learners achieve a reasonable view. In the linear and Gaussian order online learning approach, γ and C determine the maximum effect of error and the distribution of input data, respectively. We found that $\gamma = 0.7$, $C = 2$ give the best result for the online learner.

In our system, each online learner can be evaluated individually, so parallelization of each expert is sufficient for maximizing speed. The block size is another influential parameter for the calculation of the combined result. We used block sizes from 100 to 600 and found that a block size of 200 is faster and more efficient for concept drift detection. However, increasing the block size beyond this threshold has an unfavorable impact on detection.

4.1 Datasets

Our investigation datasets are divided into two overall types: benchmark and real current data. For each dataset, there are two classes for detection (normal and attack), and different types of attacks may incrementally occur and affect patterns of the new feature vector over time.

NSL-KDD. NSL-KDD is one of the public datasets applied for the data stream in intrusion detection systems. It is a new version of KDD99, which was created in 1999, giving a total of 125,000 data instances and 41 feature vectors. There are 22 kinds of attacks, each depending on a sample of the data stream feature vector.

ISCX. DoS attacks are one of the significant problems in the current era, which are considered in the ISCX dataset. Many new online classification approaches require ISCX for detecting application-layer DoS attacks. It represents 8 kinds of DoS attacks and 24 hours of network traffic.

Real data (Malicious Web Pages). Malicious web pages are another type of streaming data that changes as attackers regularly create new pages with distinctive patterns. We obtained content from malicious web pages (<https://openphish.com/>, <https://www.phishtank.com/>) and normal web pages (<https://www.alexa.com/>). All features were extracted by a web parser (Selenium) between April and August 2017. Selenium WebDriver was used as a feature extractor, using Chrome browser instances for rendering. For every URL in PhishTank and Alexa, we checked the web page because some are suspended and cannot be accessed.

Electronics. The Electronics dataset is a set of price samples about the Wales Electricity Market, created by Harries et al. [27]. Many researchers use this data to evaluate concept drift phenomena.

All detection procedures were implemented on a personal system with 8 GB of memory, Windows 10 operating system, and a 64-bit 7-core processor. Each data block is evaluated and compared to other methods using the following criteria:

- **TPR** (True Positive Rate): Number of attacks correctly detected.

$$\text{TPR} = \frac{TP}{TP + FN}$$

- **TNR** (True Negative Rate): Number of normal traffic correctly detected.

$$\text{TNR} = \frac{TN}{TN + FP}$$

- **Accuracy**: Number of data correctly detected out of all data in the stream.

$$\text{Accuracy} = \frac{TN + TP}{TP + FN + TN + FP}$$

4.2 Results

NSL-KDD. The evaluation results of the online learners and our work when the KDD data stream is received block by block are shown in Figure 7. Figure 7(a) shows that SOP and AROW move without maximal change of result and Perceptron is the most variable online learner. Among the Gaussian-order learners, SOP has the highest TPR, but AROW is increasing its TPR. Figure 7(b) shows that NAROW has the maximum value of TNR, and both Perceptron and OGD had the highest variation. Figure 7(c) shows that among all online learners, our work achieves the maximum accuracy (about 96%).

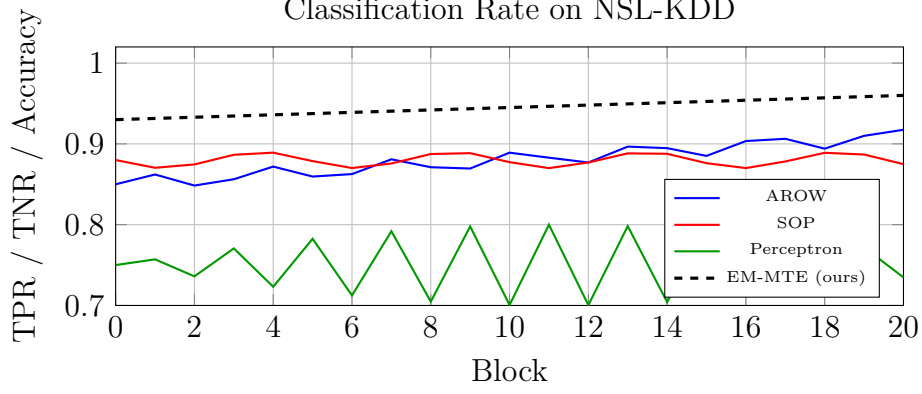


Figure 7: Classification rate of each online learning algorithm vs. our work on NSL-KDD.

ISCX. In Figure 8, another experiment uses the ISCX dataset. Initially, all online learning algorithms have a similar view of the data stream with growing TPR and TNR. CW, AROW, and SCW-I have maximal initial TPR/TNR, but their growth rate declines over time. Given these changes, the performance of our work depends on the best learner that has the minimum change, achieving about **98.8% accuracy** with AROW as anchor.

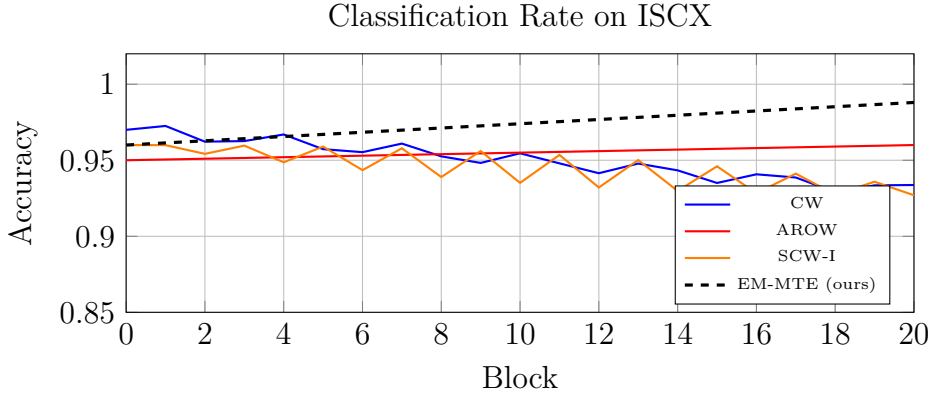


Figure 8: Classification rate of each online learning algorithm vs. our work on ISCX.

Malicious Web Pages. As shown in Figure 9, our system is analyzed based on the current web page dataset. Although compared to the best initial result, our system is not as reliable as other online learners in the early steps; its performance develops over time. In the initial steps, SOP is the most efficient detector with around **98% accuracy**. The accuracy of SOP converges to a constant value, but SCW-I, AROW, NHERD, and CW grow over time. In the final steps, our system achieves the minimum change with about **99% accuracy**.

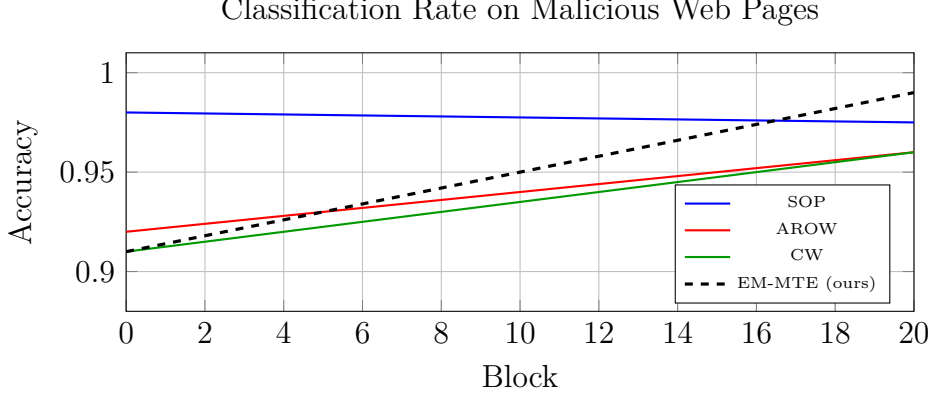


Figure 9: Classification rate of each online learning algorithm vs. our work on malicious web pages.

Electronics. In Figure 10, the TPR values for NAROW, NHERD, and AROW are the highest among other online algorithms but decline significantly over time for NHERD and AROW. Some linear-order online algorithms such as PA, PA-I, PA-II, Perceptron, and IELLIP have the highest performance among other learners, because linear-order is better than Gaussian-order algorithms when sudden and immediate drift is observed. As shown in Figure 10, the accuracy of the proposed algorithm is about **87%**, which is better than other online learners.

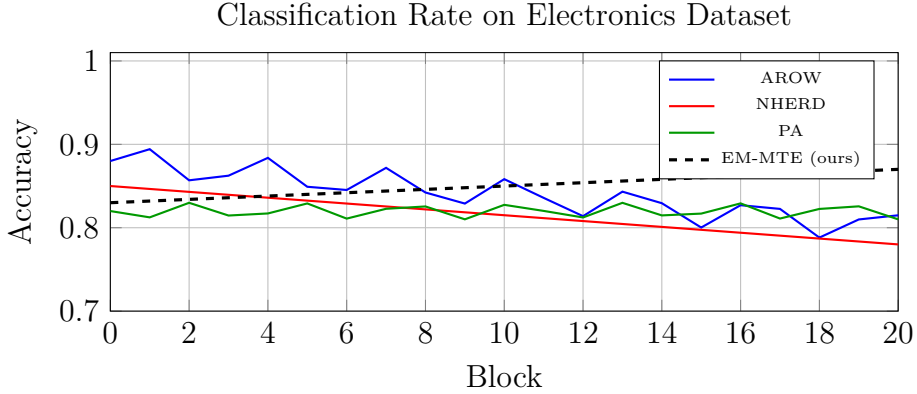


Figure 10: Classification rate of each online learning algorithm vs. our work on the Electronics dataset.

5 Conclusion

In this paper, we provide an efficient method for discovering concept drift in data streams. Our approach classifies data using an EM-based multi-task ensemble in which online learning algorithms act as shared components and drift-sensitive objectives act as tasks. To address the concept drift problem, our method consists of two parts: (1) linear-order and Gaussian-order online components, and (2) an EM fusion mechanism that learns task-specific mixture weights through latent responsibilities. Each online component considers a different type of concept drift in the data stream.

In the ensemble phase, we require each online learner’s view to achieve better prediction performance. We take a block sequence of a data stream and compute the task-

specific feedback weights by EM: the E-step estimates the posterior responsibility of each component, and the M-step updates both the task mixture weights and the shared component parameters. There is no need to store the full stream in memory; the state changes in every block iteration, and the detection procedure can be updated without full retraining. The results show that, most of the time, our method achieves the highest performance; in some cases, our method orients toward the best online learner. Also, the proposed approach is very practical in real environments.

References

- [1] B. Krawczyk, L. L. Minku, J. Gama, J. Stefanowski, and M. Woźniak, “Ensemble learning for data stream analysis: A survey,” *Information Fusion*, vol. 37, pp. 132–156, 2017.
- [2] T. Escovedo, A. Koshiyama, A. A. da Cruz, and M. Vellasco, “DetectA: Abrupt concept drift detection in non-stationary environments,” *Applied Soft Computing*, vol. 62, pp. 119–133, 2018.
- [3] D. M. Farid, L. Zhang, A. Hossain, C. M. Rahman, R. Strachan, G. Sexton, and K. Dahal, “An adaptive ensemble classifier for mining concept drifting data streams,” *Expert Systems with Applications*, vol. 40, no. 15, pp. 5895–5906, 2013.
- [4] B. Babcock, S. Babu, M. Datar, R. Motwani, and J. Widom, “Models and issues in data stream systems,” in *Proc. 21st ACM SIGMOD-SIGACT-SIGART Symp. Principles of Database Systems*, pp. 1–16, 2002.
- [5] G. J. Ross, N. M. Adams, D. K. Tasoulis, and D. J. Hand, “Exponentially weighted moving average charts for detecting concept drift,” *Pattern Recognition Letters*, vol. 33, no. 2, pp. 191–198, 2012.
- [6] P. Domingos and G. Hulten, “Mining high-speed data streams,” in *Proc. 6th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, pp. 71–80, 2000.
- [7] G. Hulten, L. Spencer, and P. Domingos, “Mining time-changing data streams,” in *Proc. 7th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, pp. 97–106, 2001.
- [8] A. Bifet and R. Gavaldà, “Adaptive learning from evolving data streams,” in *Int. Symp. Intelligent Data Analysis*, Springer, pp. 249–260, 2009.
- [9] D. Mena-Torres and J. S. Aguilar-Ruiz, “A similarity-based approach for data stream classification,” *Expert Systems with Applications*, vol. 41, no. 9, pp. 4224–4234, 2014.
- [10] M. Tennant, F. Stahl, O. Rana, and J. B. Gomes, “Scalable real-time classification of data streams with concept drift,” *Future Generation Computer Systems*, vol. 75, pp. 187–199, 2017.
- [11] J. Gama, P. Medas, G. Castillo, and P. Rodrigues, “Learning with drift detection,” in *Proc. Brazilian Symp. Artificial Intelligence*, Springer, pp. 286–295, 2004.

- [12] M. Baena-García, J. del Campo-Ávila, R. Fidalgo, A. Bifet, R. Gavaldà, and R. Morales-Bueno, “Early drift detection method,” in *Proc. 4th ECML PKDD Int. Workshop Knowledge Discovery from Data Streams*, 2006.
- [13] G. Liang, S. R. Weller, J. Zhao, F. Luo, and Z. Y. Dong, “A framework for cyber-topology attacks: Line-switching and new attack scenarios,” *IEEE Transactions on Smart Grid*, 2017.
- [14] Y. Sun, L. Teng, S. Yin, J. Liu, and H. Li, “Study a join query strategy over data stream based on sliding windows,” in *Int. Conf. Data Mining and Big Data*, Springer, pp. 334–342, 2017.
- [15] M. Torquati, G. Mencagli, M. Drocco, M. Aldinucci, T. De Matteis, and M. Dane-lutto, “On dynamic memory allocation in sliding-window parallel patterns for streaming analytics,” *The Journal of Supercomputing*, pp. 1–18, 2017.
- [16] B. Irie and S. Miyake, “Capabilities of three-layered perceptrons,” in *IEEE Int. Conf. Neural Networks*, vol. 1, pp. 641–648, 1988.
- [17] M. Zinkevich, “Online convex programming and generalized infinitesimal gradient ascent,” in *Proc. 20th Int. Conf. Machine Learning (ICML)*, pp. 928–936, 2003.
- [18] K. Crammer, O. Dekel, J. Keshet, S. Shalev-Shwartz, and Y. Singer, “Online passive-aggressive algorithms,” *Journal of Machine Learning Research*, vol. 7, pp. 551–585, 2006.
- [19] Y. Li and P. M. Long, “The relaxed online maximum margin algorithm,” in *Advances in Neural Information Processing Systems (NIPS)*, pp. 498–504, 2000.
- [20] C. Gentile, “A new approximate maximal margin classification algorithm,” *Journal of Machine Learning Research*, vol. 2, pp. 213–242, 2001.
- [21] M. Dredze, K. Crammer, and F. Pereira, “Confidence-weighted linear classification,” in *Proc. 25th Int. Conf. Machine Learning (ICML)*, pp. 264–271, 2008.
- [22] J. Wang, P. Zhao, and S. C. Hoi, “Exact soft confidence-weighted learning,” *arXiv preprint arXiv:1206.4612*, 2012.
- [23] K. Crammer, A. Kulesza, and M. Dredze, “Adaptive regularization of weight vec-tors,” in *Advances in Neural Information Processing Systems (NIPS)*, pp. 414–422, 2009.
- [24] N. Cesa-Bianchi, A. Conconi, and C. Gentile, “A second-order perceptron algo-rithm,” *SIAM Journal on Computing*, vol. 34, no. 3, pp. 640–668, 2005.
- [25] K. Crammer and D. D. Lee, “Learning via Gaussian herding,” in *Advances in Neural Information Processing Systems (NIPS)*, pp. 451–459, 2010.
- [26] L. Yang, R. Jin, and J. Ye, “Online learning by ellipsoid method,” in *Proc. 26th Annual Int. Conf. Machine Learning (ICML)*, pp. 1153–1160, 2009.
- [27] M. Harries, “Splice-2 comparative evaluation: Electricity pricing,” Technical Report, The University of New South Wales, 1999.